

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0518

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 10%

Mock Interview Questions

OpenCV Basics & Pipeline Thinking

1. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

ANSWER:

1. Debugging a Video File That Is Not Loading Properly in OpenCV

A video may fail to load in OpenCV because of an **incorrect file path, unsupported format/codec, corrupted video file, missing codec support, or incorrect OpenCV code**. A structured debugging approach helps identify the exact cause.

Step 1: Check the file path

First, verify that the video file actually exists at the specified location.

```
import os
```

```
video_path = "video.mp4"
```

```
print(os.path.exists(video_path))
```

- True → the file exists.
- False → the path or filename is incorrect.

Use the correct absolute or relative path.

Step 2: Try opening the video

```
import cv2
```

```
cap = cv2.VideoCapture(video_path)
```

```
if not cap.isOpened():
```

```
    print("Error: Cannot open video")
```

```
else:
```

```
    print("Video opened successfully")
```

If `isOpened()` returns `False`, the problem is usually related to the **path, codec, format, or OpenCV installation**.

Step 3: Check the video properties

If the video opens, check whether OpenCV can read its basic properties.

```
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
```

```
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```
fps = cap.get(cv2.CAP_PROP_FPS)
```

```
frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
print("Width:", width)
```

```
print("Height:", height)
```

```
print("FPS:", fps)
```

```
print("Frames:", frames)
```

Invalid values such as 0 for width, height, or frame count may indicate a codec or file-reading problem.

Step 4: Test frame reading

```
ret, frame = cap.read()
```

```
if not ret:
```

```
    print("Error: Frame cannot be read")
```

```
else:
```

```
    print("Frame read successfully")
```

```
    cv2.imshow("Video", frame)
```

```
    cv2.waitKey(0)
```

```
    cv2.destroyAllWindows()
```

Here, `ret` indicates whether a frame was successfully obtained.

- `ret = True` → frame is valid.
- `ret = False` → OpenCV cannot decode/read the frame.

Step 5: Check the video format and codec

Some video formats or codecs may not be supported properly by the installed OpenCV build. Try converting the video to a commonly supported format such as **MP4 with H.264** and test it again.

If the converted video works, the original problem was likely a **codec compatibility issue**.

Step 6: Check whether the video is corrupted

Open the video using another media player such as VLC. If it also fails there, the video itself may be damaged. If it works normally in another player but not in OpenCV, the problem is more likely related to **codec/backend compatibility**.

Step 7: Check the OpenCV installation

Check the installed version:

```
print(cv2.__version__)
```

If necessary, update OpenCV:

```
pip install --upgrade opencv-python
```

Step 8: Release resources properly

After processing the video, release the video object and close OpenCV windows.

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Conclusion

The debugging should proceed from **simple causes to complex causes**:

File path → Video opening → Video properties → Frame reading → Format/codecs → File corruption → OpenCV installation

This systematic process prevents random troubleshooting and helps identify whether the problem is caused by the **file, codec, system, or OpenCV code**.

2. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

ANSWER:

1. Image Appears Too Dark When Read in OpenCV

If an image appears too dark after being read in OpenCV, the problem may be caused by **low exposure, incorrect color handling, unsuitable pixel values, or an inappropriate display method**. A suitable image-processing pipeline can improve its visibility.

Step 1: Check whether the image is loaded correctly

```
import cv2

img = cv2.imread("dark_image.jpg")

if img is None:

    print("Image not loaded")

else:
```

```
    print("Image loaded successfully")
```

If img is None, the problem is the file path or image format, not brightness.

Step 2: Check the pixel intensity range

```
print("Minimum:", img.min())

print("Maximum:", img.max())
```

If most pixel values are close to 0, the image is genuinely dark.

Step 3: Convert to grayscale if required

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

This makes intensity processing easier because each pixel represents brightness from **0 (black) to 255 (white)**.

Step 4: Apply brightness adjustment

A simple way to increase brightness is:

```
bright = cv2.convertScaleAbs(img, alpha=1.0, beta=50)
```

- alpha controls contrast.
- beta controls brightness.
- Increasing beta makes dark pixels brighter.

Step 5: Improve contrast using histogram equalization

For a grayscale image:

```
equalized = cv2.equalizeHist(gray)
```

Histogram equalization spreads the intensity values over a wider range. This makes details hidden in dark regions more visible.

Step 6: Use CLAHE for better local enhancement

For images containing both dark and bright regions, CLAHE is often better:

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```
enhanced = clahe.apply(gray)
```

CLAHE improves **local contrast** without excessively brightening already bright areas.

Step 7: Display the enhanced image

```
cv2.imshow("Enhanced Image", enhanced)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Improved Processing Pipeline

Read image → Check validity → Analyze pixel intensity → Convert to grayscale → Adjust brightness → Enhance contrast → Display/save result

Justification

Brightness adjustment increases the overall intensity of dark pixels, while histogram equalization or CLAHE improves contrast and reveals details that were difficult to see. **CLAHE is generally preferable when the image contains uneven lighting**, because it enhances different regions independently instead of applying the same adjustment everywhere.

Thus, the pipeline improves visibility while reducing the risk of losing details in brighter regions.

Image & Video Processing

5. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.

ANSWER:

If a video contains many frames, processing every frame wastes **CPU time and memory** when only every 5th frame is required. The efficient approach is to read the video sequentially but process only frames whose index is divisible by 5.

Python OpenCV Solution

```
import cv2
```

```
cap = cv2.VideoCapture("video.mp4")
```

```
frame_count = 0
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    if frame_count % 5 == 0:
```

```
        # Process only every 5th frame
```

```
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
        cv2.imshow("Processed Frame", gray)
```

```
        if cv2.waitKey(1) & 0xFF == 27:
```

```
            break
```

```
    frame_count += 1
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

How It Works

Suppose the video contains frames:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10...

The program processes:

0 → 5 → 10 → 15 → 20 → ...

The condition:

if `frame_count % 5 == 0`:

selects only those frames.

Performance Consideration

Processing only every 5th frame reduces the number of frames requiring expensive operations such as **face detection, object detection, filtering, or feature extraction**.

For example, for a 30 FPS video:

- Processing every frame → **30 frames/second**
- Processing every 5th frame → approximately **6 frames/second**

Therefore, computational workload can be reduced significantly.

Accuracy Consideration

The main disadvantage is that **important events occurring between selected frames may be missed**. For example, if an object appears for only 2–3 frames, the system may never process those frames.

Therefore:

- **High processing speed required:** every 5th frame is suitable.
- **Every event must be detected:** process more frames or use adaptive frame sampling.
- **Real-time monitoring:** every 5th frame provides a good speed–accuracy trade-off when events persist across multiple frames.

Conclusion

Processing every 5th frame is an efficient **frame-sampling technique**. It reduces computational cost while retaining a representative portion of the video. However, the sampling interval should be chosen according to the speed and duration of events that the application needs to detect.

Face & Expression Recognition

12. You need to build a simple face recognition demo. Design a minimal working pipeline, explaining each stage and your design decisions.

ANSWER:

A simple face recognition system can be designed as:

**Input Image/Video → Face Detection → Face Preprocessing → Feature Extraction
→ Face Matching → Display Result**

Stage 1: Capture the Input

Use a webcam or image as the input.

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

The webcam provides frames continuously for recognition.

Stage 2: Detect the Face

Use OpenCV's Haar Cascade to locate faces.

```
face_cascade = cv2.CascadeClassifier(  
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"  
)
```

For each frame:

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
faces = face_cascade.detectMultiScale(  
    gray, scaleFactor=1.1, minNeighbors=5  
)
```

Design decision: Haar Cascade is chosen because it is simple, fast, and available directly in OpenCV. It is suitable for a basic demonstration, although modern detectors are generally more robust.

Stage 3: Preprocess the Face

Crop the detected face and resize it to a fixed size.

```
face = gray[y:y+h, x:x+w]  
face = cv2.resize(face, (100, 100))
```

Why? Every face should have the same dimensions before recognition.

Stage 4: Train a Simple Recognizer

For a minimal demo, OpenCV's LBPH recognizer can be used. Suppose we have training images of two people.

```
recognizer = cv2.face.LBPHFaceRecognizer_create()
```

```
recognizer.train(  
    training_faces,  
    training_labels  
)
```

Design decision: LBPH is lightweight and easy to implement for a small demonstration. It does not require a large deep-learning model.

Stage 5: Recognize the Face

For each detected face:

```
label, confidence = recognizer.predict(face)
```

```
print("Label:", label)
```



```
print("Confidence:", confidence)
```

The predicted label identifies the closest known person.

Stage 6: Display the Result

```
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

```
cv2.putText(
    frame,
    f"Person {label}",
    (x, y-10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (0, 255, 0),
    2
)
```

Complete Minimal Pipeline

```
import cv2
```

```
# Load face detector
```

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)
```

```
# Load trained recognizer
```

```
recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.read("face_model.yml")
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    faces = face_cascade.detectMultiScale(
        gray, 1.1, 5
    )
```

```
    for (x, y, w, h) in faces:
```

```
        face = gray[y:y+h, x:x+w]
```

```
        face = cv2.resize(face, (100, 100))
```

```
label, confidence = recognizer.predict(face)
```

```
cv2.rectangle(  
    frame, (x, y), (x+w, y+h), (0, 255, 0), 2  
)
```

```
cv2.putText(  
    frame,  
    f"Person {label}",  
    (x, y-10),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    0.8,  
    (0, 255, 0),  
    2  
)
```

```
cv2.imshow("Face Recognition", frame)
```

```
if cv2.waitKey(1) & 0xFF == 27:  
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Design Decisions

Stage	Method	Reason
Input	Webcam	Provides real-time frames
Detection	Haar Cascade	Simple and fast
Preprocessing	Grayscale + resize	Reduces complexity and standardizes input
Recognition	LBPH	Lightweight and easy for a demo
Matching	Predicted label	Identifies the closest trained face
Output	Bounding box + label	Makes the result easy to understand

Conclusion

The minimal face-recognition pipeline is **capture** → **detect** → **preprocess** → **recognize** → **display**. Haar Cascade and LBPH are appropriate for a small educational demo because they are relatively simple and computationally inexpensive. However, this should **not be treated as a production-grade recognition system**; lighting, pose, occlusion, and facial variations can significantly reduce accuracy.

Code of Conduct:

I _HARITHI S (192421151)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

HARITHI S

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately